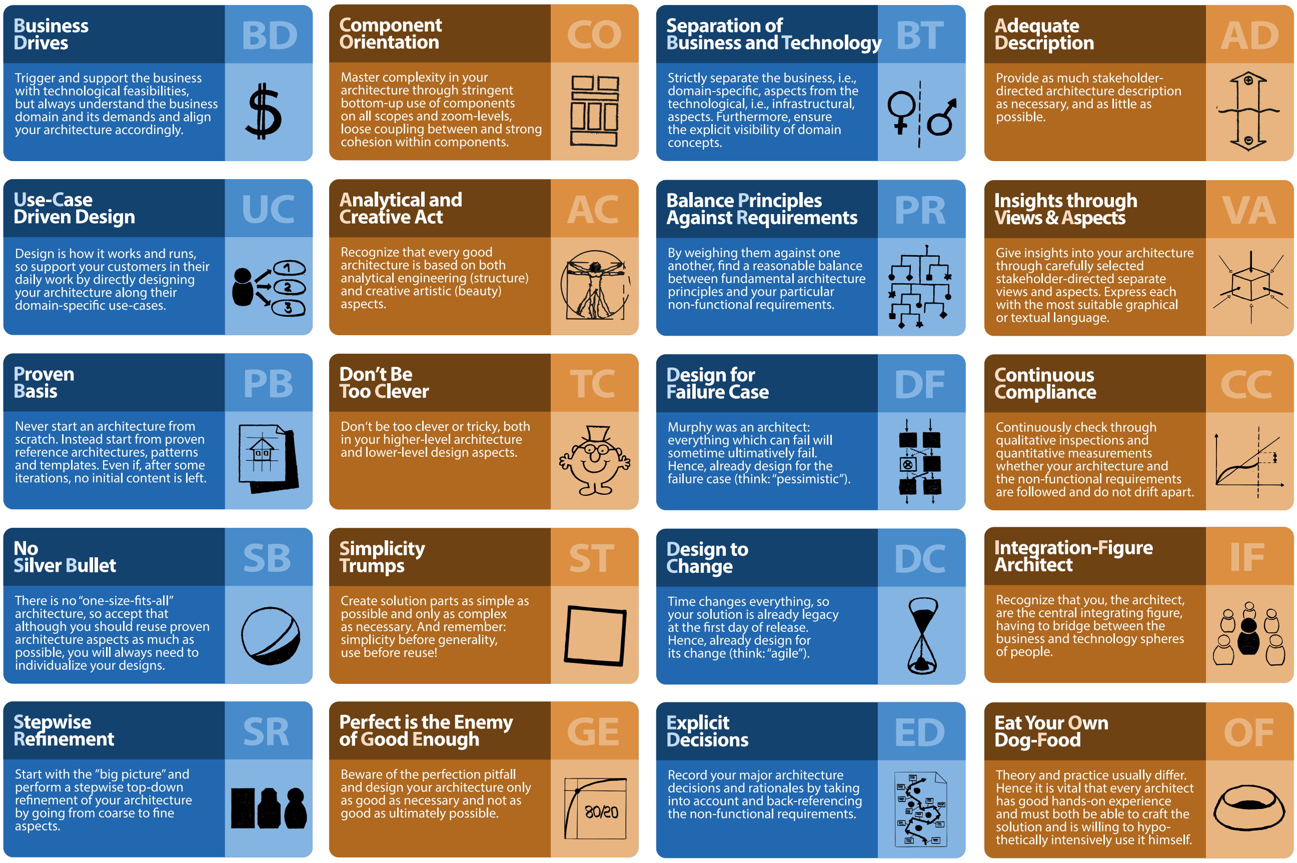
TECHNISCHE
UNIVERSITÄT
MÜNCHEN

Software Engineering in Industrial Practice
(SEIP)

Dr. Ralf S. Engelschall

Architecture Maxims



Business Drives BD
Trigger and support the business with technological feasibilities, but always understand the business domain and its demands and align your architecture accordingly.

Component Orientation CO
Master complexity in your architecture through stringent bottom-up use of components on all scopes and zoom-levels, loose coupling between and strong cohesion within components.

Separation of Business and Technology BT
Strictly separate the business, i.e., domain-specific, aspects from the technological, i.e., infrastructural, aspects. Furthermore, ensure the explicit visibility of domain concepts.

Adequate Description AD
Provide as much stakeholder-directed architecture description as necessary, and as little as possible.

Use-Case Driven Design UC
Design is how it works and runs, so support your customers in their daily work by directly designing your architecture along their domain-specific use-cases.

Analytical and Creative Act AC
Recognize that every good architecture is based on both analytical engineering (structure) and creative artistic (beauty) aspects.

Balance Principles Against Requirements PR
By weighing them against one another, find a reasonable balance between fundamental architecture principles and your particular non-functional requirements.

Insights through Views & Aspects VA
Give insights into your architecture through carefully selected stakeholder-directed separate views and aspects. Express each with the most suitable graphical or textual language.

Proven Basis PB
Never start an architecture from scratch. Instead start from proven reference architectures, patterns and templates. Even if, after some iterations, no initial content is left.

Don't Be Too Clever TC
Don't be too clever or tricky, both in your higher-level architecture and lower-level design aspects.

Design for Failure Case DF
Murphy was an architect: everything which can fail will sometime ultimatively fail. Hence, already design for the failure case (think: "pessimistic").

Continuous Compliance CC
Continuously check through qualitative inspections and quantitative measurements whether your architecture and the non-functional requirements are followed and do not drift apart.

No Silver Bullet SB
There is no "one-size-fits-all" architecture, so accept that although you should reuse proven architecture aspects as much as possible, you will always need to individualize your designs.

Simplicity Trumps ST
Create solution parts as simple as possible and only as complex as necessary. And remember: simplicity before generality, use before reuse!

Design to Change DC
Time changes everything, so your solution is already legacy at the first day of release. Hence, already design for its change (think: "agile").

Integration-Figure Architect IF
Recognize that you, the architect, are the central integrating figure, having to bridge between the business and technology spheres of people.

Stepwise Refinement SR
Start with the "big picture" and perform a stepwise top-down refinement of your architecture by going from coarse to fine aspects.

Perfect is the Enemy of Good Enough GE
Beware of the perfection pitfall and design your architecture only as good as necessary and not as good as ultimately possible.

Explicit Decisions ED
Record your major architecture decisions and rationales by taking into account and back-referencing the non-functional requirements.

Eat Your Own Dog-Food OF
Theory and practice usually differ. Hence it is vital that every architect has good hands-on experience and must both be able to craft the solution and is willing to hypothetically intensively use it himself.

In IT Architecture, one follows **Architecture Maxims**, which are basic guidelines. One knows 20 maxims. The architect should always follow the maxims and never break them.

Note: **Proven Basis** and **No Silver Bullet** say that one has to start an architecture always on a proven basis (e.g., a reference architecture), however, at the same time, it has to be clear that one cannot use these 1:1, but always first have to adjust it.

Note: **Stepwise Refinement** and **Component Orientation** say that regarding time (and for reasons of risk minimization), one always goes from the coarse to the fine, while the results show a stringent component-orientation, where small components are hierarchically integrated into larger components.

Note: while, as an IT architect, one just has to accept all maxims, **Simplicity Trumps** is from another quality: nothing in IT is really easy. When something looks simple, one just doesn't understand enough about it. Or someone really invested a lot just to make it look simple. **Simplicity Trumps** means precisely this: make inherently complex things simple again.

Questions

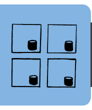
- ? For reasons of risk minimization, how should the IT architect at **Stepwise Refinement** always proceed step-by-step?

ARCHITECTURE FUNDAMENTALS

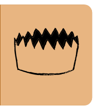
Architecture Principles

TECHNISCHE UNIVERSITÄT MÜNCHEN

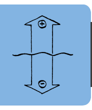
FL Factual **Locality**
Resources are as spatially and temporarily local-scoped to solution components as possible



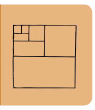
ES Exclusive **Sovereignty**
Exclusive resource sovereignty by the enclosing component



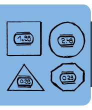
CA Contextual **Adequacy**
Neither insufficient nor exaggerated solutions for each context



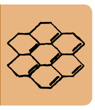
SP Solution-oriented **Proportionality**
Good expected proportionality in each solution context



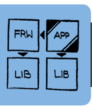
HC Holistic **Consistency**
Full consistency across all aspects of a solution



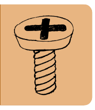
SH Structural **Homogeneity**
Maximum homogeneity in the structure of a solution



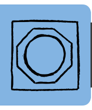
CR Constructional **Reusability**
High reuse of proven structural components and partial solutions



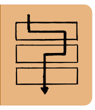
FS Fulfilled **Standards**
Compliancy to standards as much as possible, as long as the benefits predominate the drawbacks



FA Functional **Abstraction**
Suitable level of abstraction across all functional aspects of a solution

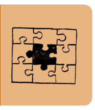


FT Functional **Traceability**
Suitable traceability across all functional aspects of a solution



CI Communicative **Interoperability**
Maximum interoperability in communication between solutions

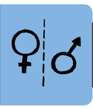
EH Environmental **Harmony**
Maximum harmony in the integration of the solution with its environment



AR Avoided **Redundancy**
Minimum total number of copies of a single resource

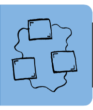
MS Minimum **Special-Cases**
Minimum total number of special-cases in a solution

LS Logical **Separation**
Separation of concerns between the components of a solution

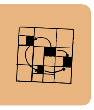


SM Structural **Modularity**
Splitting of a solution into manageable structural components

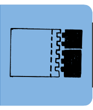
LC Loose **Coupling**
Loose coupling in communication and referencing between solution components



SC Strong **Cohesion**
Strong relationship between functionalities within a single solution component

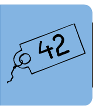


OE Open **Extensibility**
Solution components can be extended by third-parties at fixed interfaces



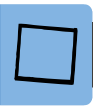
CC Closed **Changeability**
Solution components are protected against direct change by third-parties

UI Unique **Identification**
Unique identification of all components of a solution

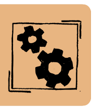


UA Uniform **Addressing**
Uniform addressing of all components of a solution

OS Overall **Simplicity**
All design aspects of a solution are as simple as possible and only as complicated as necessary

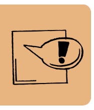


EC Encapsulated **Complexity**
Complex related aspects of a solution are encapsulated into a single responsible component

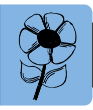


LA Least **Astonishment**
All design aspects of a solution are as little astonishing as possible and only as esoteric as necessary

SD Self **Documentation**
All design aspects of a solution are preferably self-documenting



OD Operational **Delight**
The solution provides users true delight even on long-term operation



AA Artistic **Aesthetics**
The solution has holistic aesthetics and artistic love in details

Intellectual Content: Version 1.0.15 (2022-11-05), Authored 2010-2022 by Dr. Ralf S. Engelschall
Graphical Illustration: Version 1.0.15 (2018-08-10), Copyright © 2011-2018 Dr. Ralf S. Engelschall <http://engelschall.com>, All Rights Reserved.
Unauthorized Reproduction Prohibited. Licensed to Technische Universität München (TUM) for reproduction in Computer Science lecture contexts only.

In IT Architecture, one follows **Architecture Principles**, which summarize basic principles and procedures. One knows 28 principles that can be grouped into 14 pairs since always two principles are very close regarding the content. The architect should follow the principles in general, but he may violate them as long as he has a good reason for it. The best reason would be a particular project-specific requirement.

Note: The principle **Logical Separation** (aka **Separation of Concern**) is one of the most important, since from it several other principles almost automatically follow, including, e.g., **Structural Modularity**.

Note: The principles **Loose Coupling** and **Strong Cohesion** are known in the literature as the combined principle “Loose Coupling, Strong Cohesion.” The principles **Open Extensibility** and **Closed Changeability** are known in the literature as the combined principle “Open-Close.”

Note: The principle **Overall Simplicity** is one of the hardest to implement because nothing in IT is really easy. Everything only looks simple as long as one does not have enough understanding about it. After that, one first has to make it “simple” painstakingly. That’s the art of architecture: simplify difficult things! If something cannot be simplified further and still has a certain complexity, following the principle **Encapsulated Complexity**, one at least can try to shadow it.

Questions

? List at least 4 essential **Architecture Principles**!

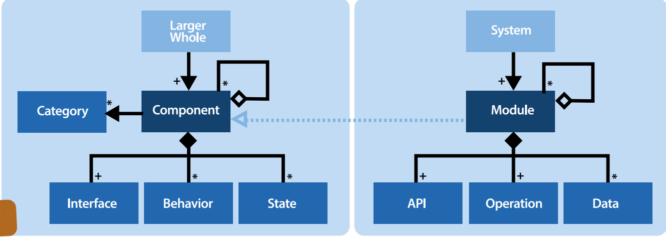
Component Design

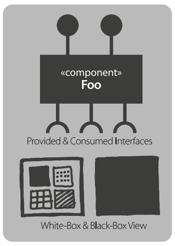
Definition of a Component (of a Larger Whole):
a **know-how encapsulating**, potentially **reusable** and **substitutable unit of hierarchical composition** with **explicit context dependencies**, which **hides the complexity** of its optional **behavior** and **state** realization behind **small contractually specified interfaces**, defines its **added value** in terms of **provided** and **consumed** interfaces and optionally belongs to zero or more **categories** of similar units.

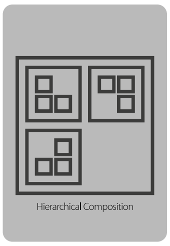
Definition of a Module (of a System):
a know-how encapsulating, potentially reusable and substitutable **source-code** unit of hierarchical composition with explicit context dependencies which hides the complexity of its **operation** and **data implementation** behind small contractually specified **Application Programming Interfaces (API)**, defines its added value in terms of provided and consumed **APIs** and optionally belongs to zero or more categories of similar units.

Example Categories of Components:
- Namespace
- Directory, File
- Configuration, Section, Directive
- Host, Virtual Machine, Container
- Process Group, Process, Thread
- Application, Microservice, Program
- Package, Class, Function
- Database, Schema, Table, Record
- Datamodel, Entity Group, Entity
- User Interface, Dialog, Widget

Any group of anything!







How to find Components (or Modules)?

DCA Domain Concept Abstraction
Model domain concepts as entity components and then group at higher levels.

SOC Separation of Concerns
Build components for clearly distinct concerns.

USE Reusability Potential
Decide on components based on their reusability potential.

UCC Use-Case Clustering
Build domain components for each use-case or each logical use-case cluster.

SRP Single Responsibility Principle
Build components for clearly distinct responsibilities.

DCC Divide & Conquer Complexity
Master overall complexity by splitting larger things into smaller things.

DDD Domain-Driven Design
Model domain "Bounded Contexts" through DDD and derive components from them.

CNC Coupling and Cohesion
Decide on components based on their loose coupling and strong cohesion.

CCC Cross-Cutting Concerns
Build common cross-cutting concerns as cross-cutting components.

OOD Object-Oriented Design
Model Object-Oriented Design entities (and/or OOP constructs) as components.

DEP Dependency Encapsulation
Decide on components based on their encapsulation of dependencies.

PAT Architecture Patterns
Build inner components to comply to outer structure, slicing and clustering architectures.

• • •

Software Architecture is all about **Components** and **Interfaces**. Therefore, **Component Design** is a central task of the architect.

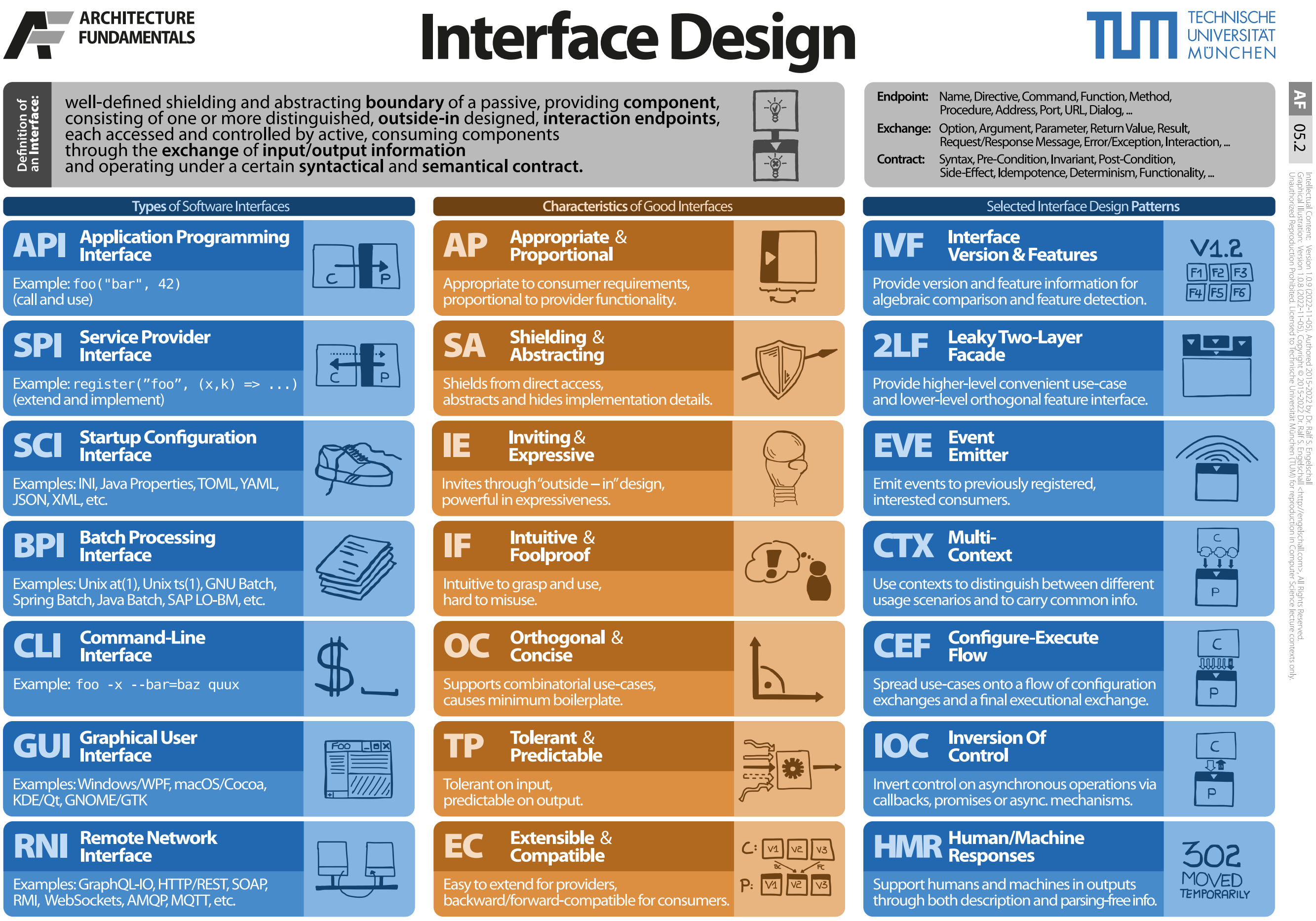
A component **encapsulates** a certain **know-how**, is **potentially reusable** and **replaceable**. A component has a **behaviour** and a **state** and hides the internal complexity of both behind "small" **contractual interfaces**. It provides its added value through the difference between provided and consumed interfaces. It can be considered as a **Whitebox** or as a **Blackbox**, depending on whether the internal details can be viewed from outside or not. Components are arranged hierarchically, may belong to specific **categories** and have **explicit dependencies** among each other.

A distinction is made between the more general concept of **Component** ("any group of anything") and the more specific concept of the (via Source code defined) **Module**.

Components can be found in many different ways. Most of them are directly derived from existing methods, principles, or patterns. The two most important ways for a component cut in practice are: **Separation of Concerns** (which unique concern or task has the component?) and **Single Responsibility Principle** (what is the unique responsibility of the component?).

Questions

- ? List at least 7 properties/aspects which a Component has!
- ? What are the two most important ways to find a component cut in practice?



ARCHITECTURE FUNDAMENTALS

Interface Design

TECHNISCHE UNIVERSITÄT MÜNCHEN

AF 05.2

Definition of an Interface: well-defined shielding and abstracting **boundary** of a passive, providing **component**, consisting of one or more distinguished, **outside-in** designed, **interaction endpoints**, each accessed and controlled by active, consuming components through the **exchange** of **input/output information** and operating under a certain **syntactical** and **semantical contract.**

Endpoint: Name, Directive, Command, Function, Method, Procedure, Address, Port, URL, Dialog, ...

Exchange: Option, Argument, Parameter, Return Value, Result, Request/Response Message, Error/Exception, Interaction, ...

Contract: Syntax, Pre-Condition, Invariant, Post-Condition, Side-Effect, Idempotence, Determinism, Functionality, ...

Types of Software Interfaces

API **Application Programming Interface**
Example: `foo("bar", 42)` (call and use)

SPI **Service Provider Interface**
Example: `register(”foo”, (x,k) => ...)` (extend and implement)

SCI **Startup Configuration Interface**
Examples: INI, Java Properties, TOML, YAML, JSON, XML, etc.

BPI **Batch Processing Interface**
Examples: Unix at(1), Unix ts(1), GNU Batch, Spring Batch, Java Batch, SAP LO-BM, etc.

CLI **Command-Line Interface**
Example: `foo -x --bar=baz quux`

GUI **Graphical User Interface**
Examples: Windows/WPF, macOS/Cocoa, KDE/Qt, GNOME/GTK

RNI **Remote Network Interface**
Examples: GraphQL-IO, HTTP/REST, SOAP, RMI, WebSockets, AMQP, MQTT, etc.

Characteristics of Good Interfaces

AP **Appropriate** & **Proportional**
Appropriate to consumer requirements, proportional to provider functionality.

SA **Shielding** & **Abstracting**
Shields from direct access, abstracts and hides implementation details.

IE **Inviting** & **Expressive**
Invites through "outside – in" design, powerful in expressiveness.

IF **Intuitive** & **Foolproof**
Intuitive to grasp and use, hard to misuse.

OC **Orthogonal** & **Concise**
Supports combinatorial use-cases, causes minimum boilerplate.

TP **Tolerant** & **Predictable**
Tolerant on input, predictable on output.

EC **Extensible** & **Compatible**
Easy to extend for providers, backward/forward-compatible for consumers.

Selected Interface Design **Patterns**

IVF **Interface Version & Features**
Provide version and feature information for algebraic comparison and feature detection.

2LF **Leaky Two-Layer Facade**
Provide higher-level convenient use-case and lower-level orthogonal feature interface.

EVE **Event Emitter**
Emit events to previously registered, interested consumers.

CTX **Multi-Context**
Use contexts to distinguish between different usage scenarios and to carry common info.

CEF **Configure-Execute Flow**
Spread use-cases onto a flow of configuration exchanges and a final executional exchange.

IOC **Inversion Of Control**
Invert control on asynchronous operations via callbacks, promises or async. mechanisms.

HMR **Human/Machine Responses**
Support humans and machines in outputs through both description and parsing-free info.

An **interface** is a **well-defined**, **shielding**, **abstracting boundary** of a passive providing **component**, which consists of one or more clearly distinguishable **interaction endpoints**.

At each interaction endpoint, an active, consuming component is accessed through the **exchange** of **input/output information** and is operated under a specific **syntactical** and **semantical contract**.

There are numerous kinds of interfaces, all of which meet this definition. In addition, "good" interfaces have specific Properties/Characteristics. The four of the best properties are: **Proportional** (the interface is smaller and in size proportional to the functionality behind it), **Expressive** (the interface provides a powerful programming model), **Orthogonal** (the interface allows combinatorial Use-Cases), and **Concise** (the interface generates little "Boilerplate Code" during use).

There are numerous software patterns for interfaces. An interesting pattern is the **Leaky Two-Layer Facade**, in which a library has two interfaces: an upper, convenient, and Use-Case-related interface and a lower, orthogonal Feature-related interface. The trick is that the upper interface is implemented by the lower interface only and that the lower interface "shines through" ("leaky" or Open Layering).

Questions

- ? List at least 8 properties/aspects which define an **Interface**!
- ? List at least 4 properties/characteristics of **good Interfaces**!